

Programming Assignment 2

Pattern Recognition and Neural Networks, 2026

Department of Electrical Communication Engineering, IISc

Prathosh A P

Overview & General Instructions

In this assignment, you will transition from classical machine learning to deep learning using **PyTorch**.

Rules of Engagement:

- **No Black Boxes:** You are forbidden from treating PyTorch as a black box. You must extract intermediate tensors, track gradient flow, and mathematically justify your architectural choices.
- **No High-Level Wrappers:** Do not use PyTorch Lightning, FastAI, or similar abstractions. You must write your own training loops.
- **Pre-trained Models:** Forbidden, except explicitly where authorized in Phase 6.
- **Deliverables:** Submit a 4-page maximum PDF report containing the requested plots, mathematical explanations, and metrics, along with your heavily commented Python/Jupyter code.

The Datasets & Mandatory Splits

You will evaluate your architectures across three real-world data modalities. Before beginning, you must strictly partition all datasets into **Training (70%)**, **Validation (15%)**, and **Testing (15%)**. *All hyperparameter tuning must be done on the Validation set. The Test set must be evaluated only once.*

1. **Delhi Air Quality Time-Series:** Hourly meteorological and pollution data (PM2.5, temp, wind).
Split Rule: **Do not shuffle.** Split strictly chronologically to prevent temporal leakage.
Dataset Link: <https://www.kaggle.com/datasets/deepaksirohiwal/delhi-air-quality>
2. **PlantVillage Crop Disease:** High-resolution images of healthy and diseased crop leaves.
Split Rule: Use stratified random sampling to maintain class balance.
Dataset Link: <https://www.kaggle.com/datasets/abdallahalidev/plantvillage-dataset>
3. **APTOS 2019 Blindness Detection:** Retinal fundus images graded for Diabetic Retinopathy.
Split Rule: Use stratified random sampling to preserve the extreme class imbalance.
Dataset Link: <https://www.kaggle.com/c/aptos2019-blindness-detection/data>

1 Phase 1: The MLP Baseline & Gradient Flow (Air Quality)

- 1.1 Flattened Temporal MLP (Regression):** Take the past 72 hours of meteorological data, flatten the temporal dimension into a single feature vector, and train a 3-layer Multi-Layer Perceptron (MLP) to predict the $t + 24$ PM2.5 value. Report the Mean Squared Error (MSE) on the test set.
- 1.2 The Vanishing Gradient Proof:** Re-initialize your MLP, but replace all ReLU activations with Sigmoid activations. Write a PyTorch `backward_hook` to capture the `gradients of the weights` in the first, second, and third linear layers during the first epoch. Plot the L_2 norm of these gradients. Mathematically explain the observed decay.
- 1.3 Imbalanced Threshold Classification:** Convert the target variable into a binary classification task: 1 if $\text{PM2.5} > 200$ (Hazardous), else 0. Train an MLP using standard `BCELoss`, then train a second model using `BCEWithLogitsLoss` with the `pos_weight` parameter mathematically scaled to the class imbalance. Compare their `Precision-Recall curves`.

2 Phase 2: CNNs & Spatial Hierarchies (PlantVillage)

- 2.4 CNN from Scratch & Receptive Fields:** Build a CNN from scratch using `nn.Conv2d`, `nn.MaxPool2d`, and `nn.BatchNorm2d`. For a 128×128 input image, programmatically print the exact shape of the tensor as it passes through each layer. Calculate and report the `theoretical receptive field` of a single activation in your final feature map.
- 2.5 CNN Regression Head:** Modify your CNN architecture to perform regression to predict a continuous "Severity Percentage" (`synthesize this target by generating a bounding box ratio of necrotic tissue vs. healthy leaf`). Change the final loss function to MSE. Train the network and report the Mean Absolute Error (MAE).
- 2.6 Translation Invariance:** Take 100 test images and evaluate your trained CNN classifier to get a baseline accuracy. Next, write a `torchvision.transforms` function to artificially shift all pixels in these test images 5 pixels to the right and 5 pixels down (padding with zeros). Re-evaluate the accuracy. `Document the performance drop and explain why standard max-pooling does not provide perfect mathematical translation invariance.`

3 Phase 3: RNNs & Sequential Memory (Air Quality)

- 3.7 Vanilla RNN from Scratch:** Do not use `nn.RNN`. Implement a vanilla Recurrent Neural Network cell from scratch using only `nn.Linear` layers and a `tanh` activation inside a Python `for` loop over the time dimension. Train it on the 72-hour historical sequences to predict the continuous PM2.5 value.
- 3.8 BPTT Decay:** Using your vanilla RNN, pass a 100-step sequence through the `untrained network`. Compute the loss and call `.backward()`. `Extract the gradient of the loss with respect to the hidden state at  $t = 100$ ,  $t = 50$ , and  $t = 0$ .` Plot these gradient magnitudes to empirically demonstrate the vanishing gradient problem over time.
- 3.9 Exploding Gradients:** Artificially increase the learning rate of your vanilla RNN and remove any gradient clipping. Train the model until the loss evaluates to `NaN`. Extract the weight matrices exactly one step before the crash and compute their maximum singular value. Explain how this numerical value mathematically guarantees an exploding gradient.

4 Phase 4: LSTMs, GRUs, & Context (Air Quality)

- 4.10 LSTM Gradient Rescue:** Replace your vanilla RNN with PyTorch's `nn.LSTM`. Repeat the exact experiment from Question 3.8 (extracting gradients at $t = 100, 50, 0$). Plot the LSTM gradient magnitudes alongside the vanilla RNN gradients. Explain which specific mathematical gate mechanism prevents the gradient from vanishing to zero at $t = 0$.
- 4.11 GRU vs. LSTM Efficiency:** Train a standard `nn.GRU` on the same regression task. Write a script to iterate through the model's `state_dict()` and compute the exact number of trainable parameters for both the LSTM and the GRU. Compare their final validation MSE and training time per epoch.
- 4.12 Sequence-to-Sequence Forecasting:** Modify your LSTM to act as an Encoder-Decoder. Use the 72-hour history to encode a context vector, then use an autoregressive LSTM decoder to predict the full 24-hour trajectory (hours 1 through 24). Plot a test sample showing the ground truth sequence overlaid with your model's 24-step prediction.

5 Phase 5: Transformers & Attention

- 5.13 Scaled Dot-Product Attention:** Do not use `nn.Transformer`. Implement a single Scaled Dot-Product Attention head from scratch using PyTorch tensor multiplications. Apply it to the 72-hour meteorological sequence. Extract the 72×72 attention weight matrix for a specific test sample and plot it as a heatmap.
- 5.14 The Necessity of Positional Encoding:** Train a simple Transformer Encoder on the time-series data *without* adding any positional encodings. Report the validation loss. Then, add standard sinusoidal positional encodings to the input embeddings and retrain. Explain mathematically why the first model fundamentally failed to capture temporal trends.
- 5.15 Vision Transformer (ViT)** Apply the Transformer architecture to the PlantVillage image dataset. Write a function that takes a 128×128 image, slices it into a grid of 16×16 non-overlapping patches, and flattens each patch into a 1D sequence. Pass this sequence of patches through your Transformer Encoder to classify the disease. Compare its parameter efficiency to your CNN from Phase 2.

6 Phase 6: Advanced Vision & Imbalance (APTOS 2019)

- 6.16 Transfer Learning & Freezing:** Training your custom CNN from scratch on the APTOS dataset will likely result in mode collapse (predicting only Class 0). Load a pre-trained `torchvision.models.resnet18`. Freeze all convolutional layers. Replace the final fully connected layer to output 5 classes. Train *only* this new head. Report the exact number of trainable versus frozen parameters, and plot the validation loss curve.
- 6.17 Ordinal Imbalance & Focal Loss:** The severity scale (0 to 4) is ordinal, and heavily skewed toward Class 0. Implement Focal Loss from scratch (do not use a library implementation) to force the model to focus on the hard, minority classes. Evaluate your model using Cohen's Quadratic Weighted Kappa (QWK). Explain mathematically why standard Cross-Entropy accuracy is a deeply misleading metric for this specific dataset.